

Computer Organization & Architecture

Unit- 7

Pipeline And Vector Processing(Part - 1)

Prepared By: Sweetu D. Sureja

Information Technology Department

V.V.P. Engineering College, Rajkot.

FLYNN'S CLASSIFICATION

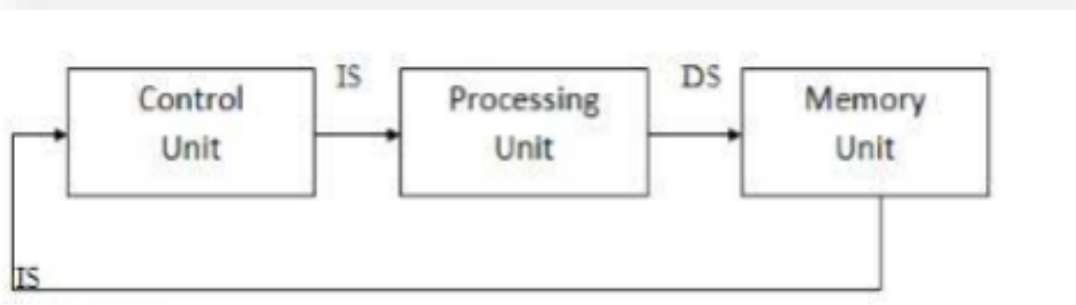
- Flynn's Classification is the most popular taxonomy of computer architecture, proposed by **Michael J. Flynn** in 1966 based on **number of instructions and data**
- It is based on the notion of **a stream of information**. Two types of information flow into a processor: **instructions** and **data**.
- The **Instruction Stream** is defined as the *sequence of instructions* executed by the processing unit
- The **Data Stream** is defined as *the sequence of data* including inputs, partial, or temporary results, called by the instruction stream

Flynn's Classification

- ♦ **Flynn's classification** divides computers into four major groups as follows:
 - ♦ Single instruction stream, single data stream (SISD)
 - ♦ Single instruction stream, multiple data stream (SIMD)
 - ♦ Multiple instruction stream, single data stream (MISD)
 - ♦ Multiple instruction stream, multiple data stream (MIMD)

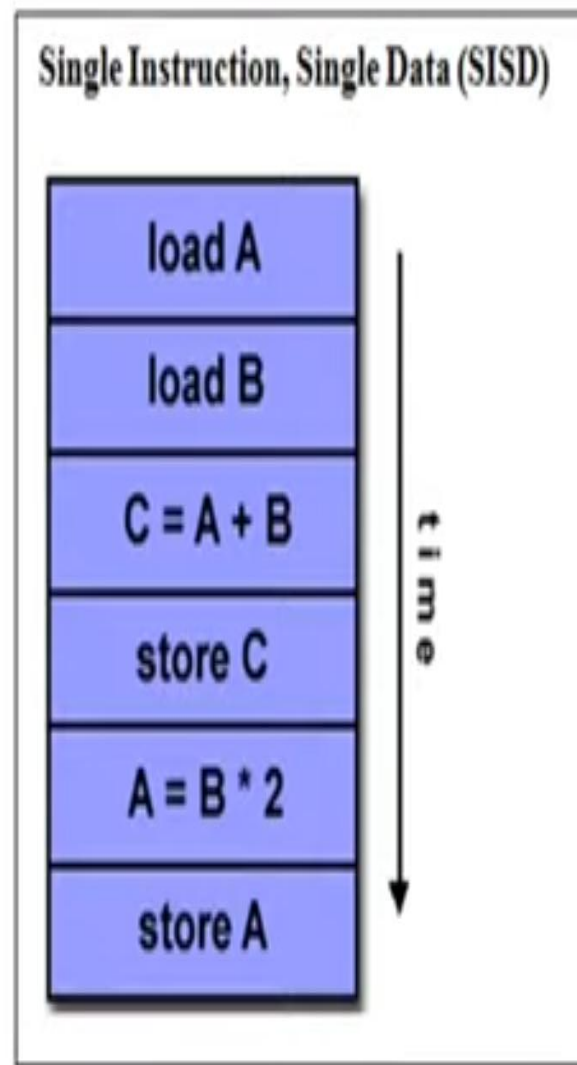
		Data Stream	
		Single	Multiple
Instruction Stream	Single	SISD	SIMD
	Multiple	MISD	MIMD

1) Single instruction stream, single data stream (SISD)



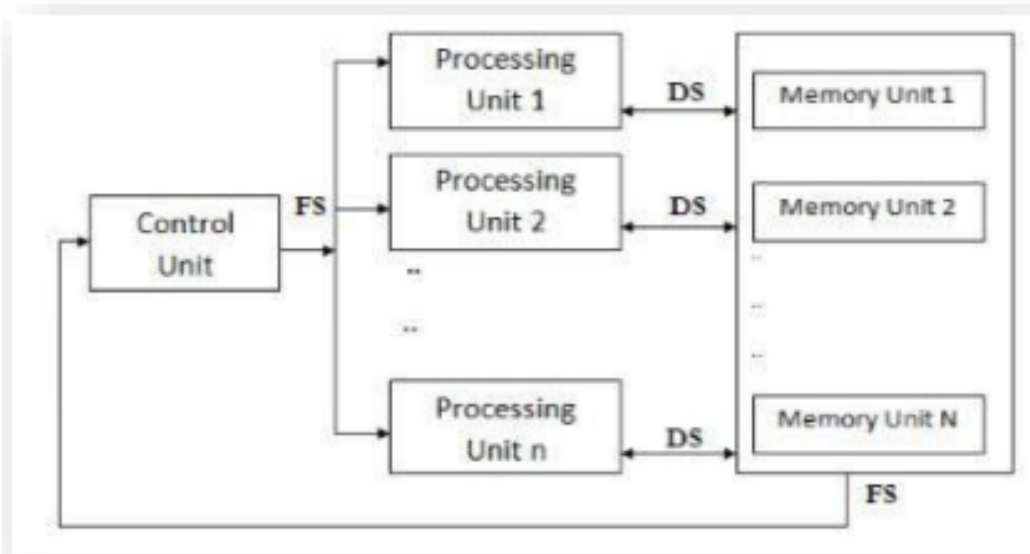
- ♦ SISD represents the organization of a single computer containing a control unit, a processor unit, and a memory unit.
- ♦ Instructions are executed sequentially and the system may or may not have internal parallel processing capabilities.

SISD Computers.....



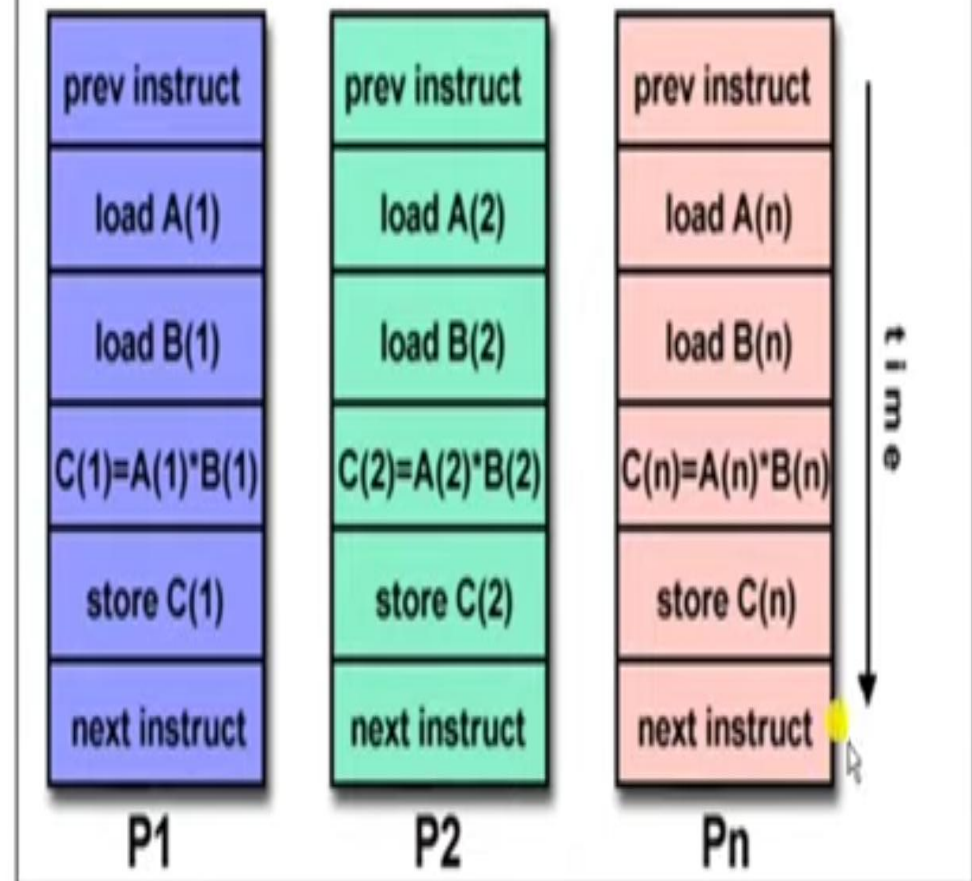
2) Single instruction stream, multiple data stream (SIMD)

SIMD Computers...



- ◆ SIMD represents an organization that includes many processing units under the supervision of a common control unit.
- ◆ All processors receive the same instruction from the control unit but operate on different items of data.
- ◆ The shared memory unit must contain multiple modules so that it can communicate with all the processors simultaneously.

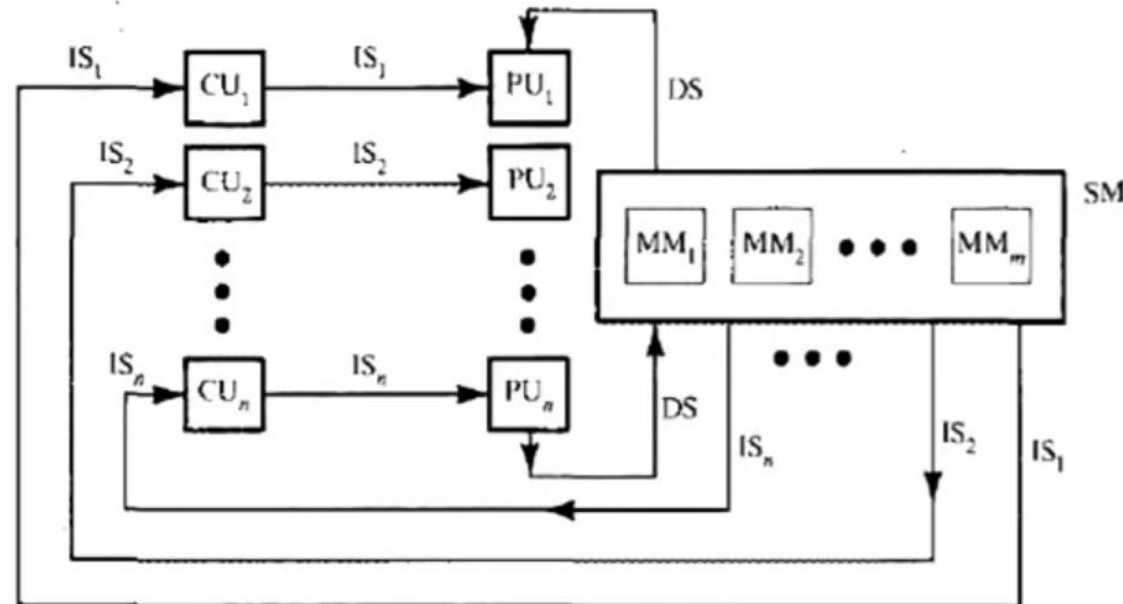
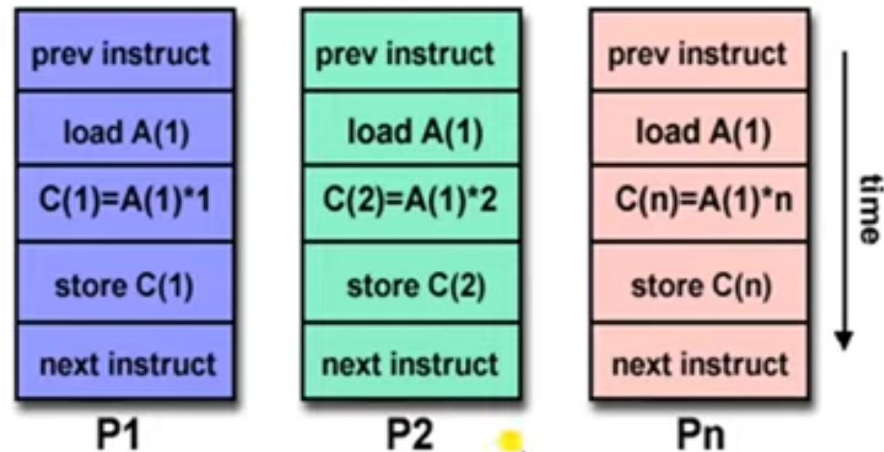
Single Instruction, Multiple Data (SIMD)



MISD Computers

- ❑ In MISD, multiple instructions operate on single data stream
- ❑ **Multiple instruction** : Each processing unit may be executing a different instruction stream
- ❑ **Single data**: Every processing unit can operate on a same data element
- ❑ Machines built using the MISD model are practically not useful in most of the application, a few machines are built, but none of them are available commercially.

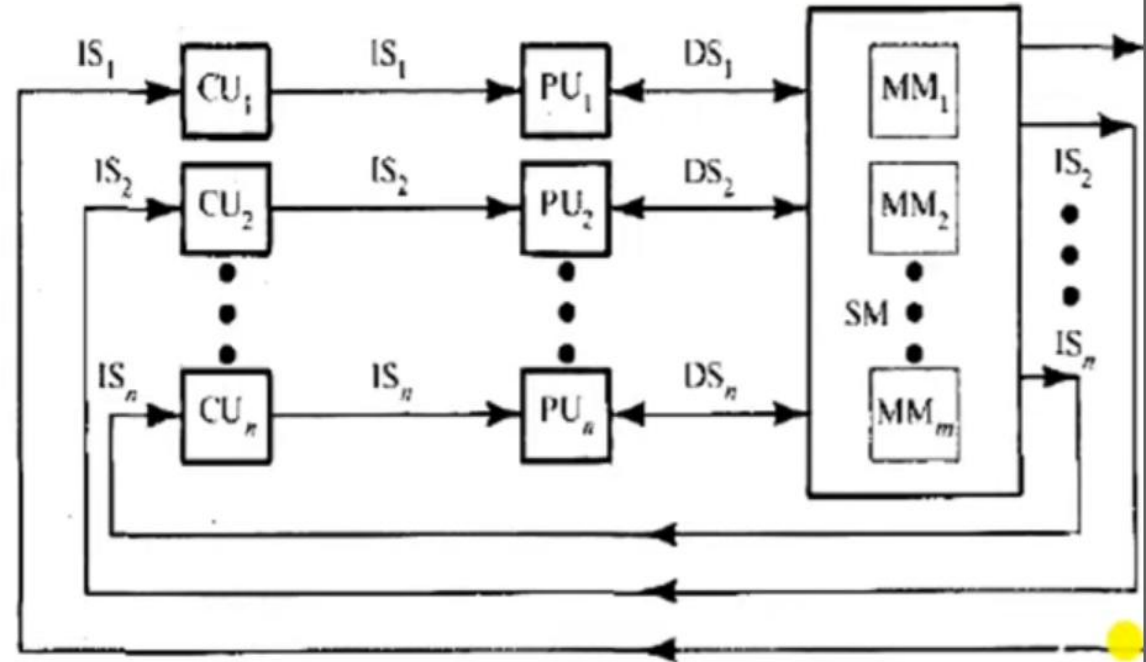
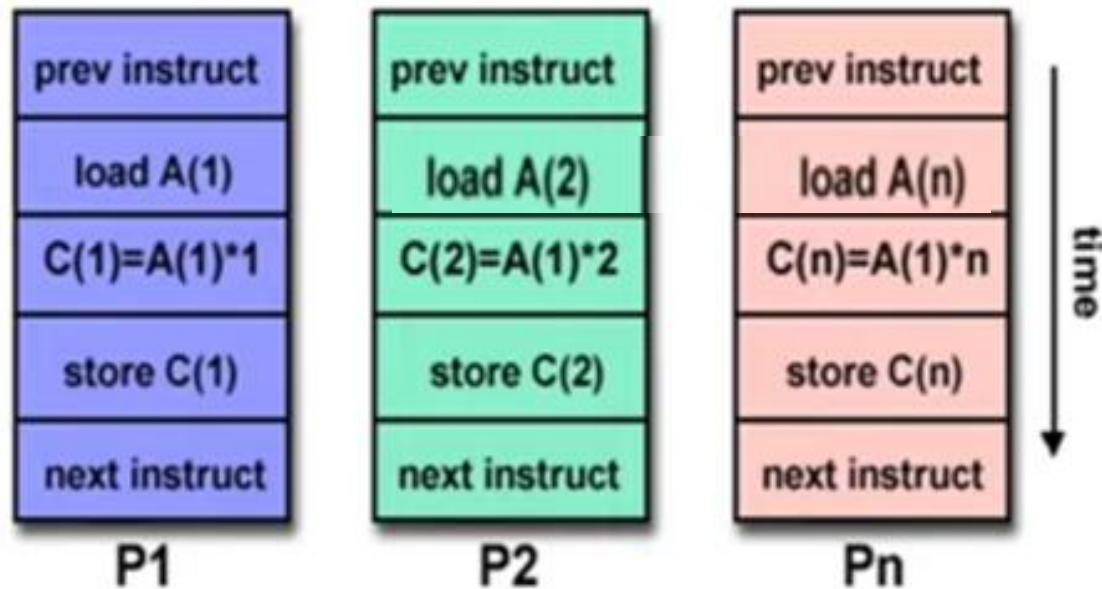
Multiple Instruction, Single Data (MISD)



MIMD Computers

- An MIMD system is capable of executing multiple instructions on multiple data sets
- **Multiple Instruction:** Every processor may be executing a different instruction stream
- **Multiple Data:** Every processor may be working with a different data stream
- MIMD systems are parallel computers capable of processing several programs simultaneously

Multiple Instruction, Multiple Data (MIMD)

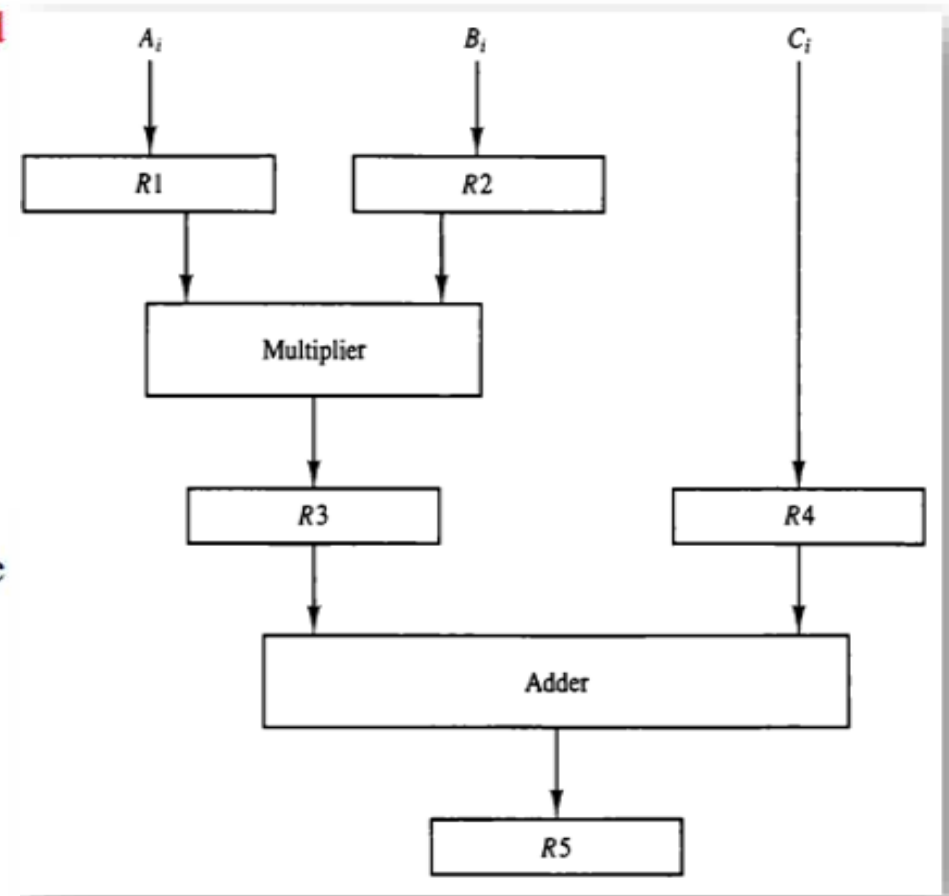


Parallel Processing

- ♦ Parallel processing is a technique that are used to **provide simultaneous data-processing tasks** for the purpose of increasing the computational speed of a computer system.
- ♦ Instead of processing each instruction sequentially as in a conventional computer, a parallel processing system is able to **perform concurrent data processing** to achieve faster execution time.
- ♦ For example, **while an instruction is being executed** in the ALU, the **next instruction can be read** from memory.
- ♦ The system may have two or more ALUs and be able to **execute two or more instructions at the same time**.
- ♦ Furthermore, the system may **have two or more processors operating concurrently**.
- ♦ The purpose of parallel processing is to **speed up the computer processing capability** and increase its throughput.

Pipelining

- ◆ Pipelining is a technique of decomposing a sequential process into sub operations, with each sub process being executed in a special dedicated segment that operates concurrently with all other segments.
- ◆ Suppose that we want to perform the combined multiply and add operations with a stream of numbers.
- ◆ $A_i * B_i + C_i$ for $i = 1, 2, 3, \dots, 7$
- ◆ Each sub operation is to be implemented in a segment
- ◆ The sub operations performed in each segment of the pipeline are as follows:
 - ◆ $R1 \leftarrow A_i, R2 \leftarrow B_i$ Input A_i and B_i
 - ◆ $R3 \leftarrow R1 * R2, R4 \leftarrow C_i$ Multiply and input C_i
 - ◆ $R5 \leftarrow R3 + R4$ Add C_i to product



- ♦ The five registers are loaded with new data every clock pulse.

- ♦ The **first clock pulse** transfers A_1 and B_1 into R1 and R2.

$$R1 \leftarrow A_1 \quad R2 \leftarrow B_1$$

- ♦ The **second clock pulse** transfers the product of R1 and R2 into R3 and C_1 into R4.
 A_2 and B_2 into R1 and R2.

$$R3 \leftarrow R1 * R2, \quad R4 \leftarrow C_1$$

$$R1 \leftarrow A_2 \quad R2 \leftarrow B_2$$

- ♦ The **third clock pulse** operates on all three segments simultaneously. It places A_3 and B_3 into R1 and R2, transfers the product of R1 and R2 into R3, transfers C_2 into R4, and places the sum of R3 and R4 into R5.

$$R5 \leftarrow R3 + R4$$

$$R3 \leftarrow R1 * R2, \quad R4 \leftarrow C_2$$

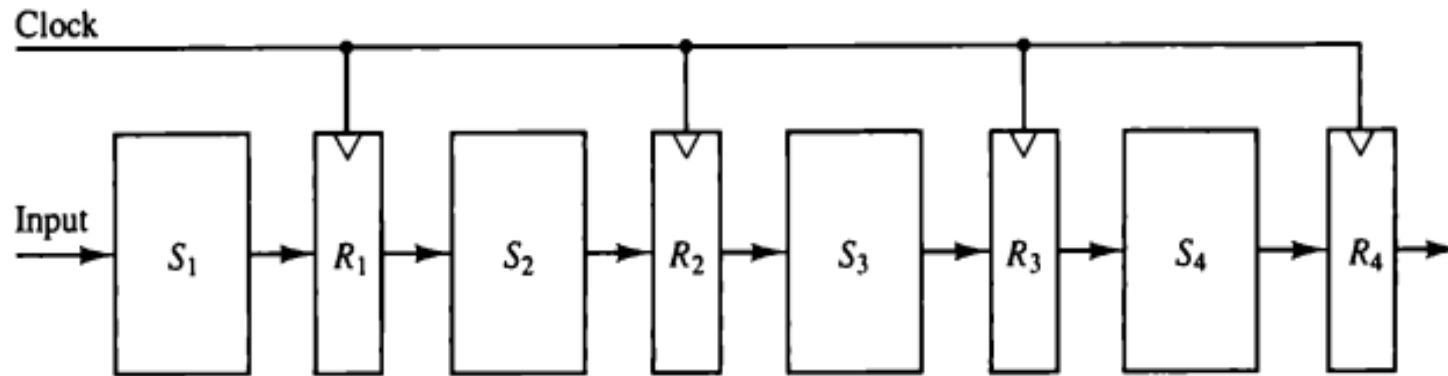
$$R1 \leftarrow A_3 \quad R2 \leftarrow B_3$$

- ♦ It takes **three clock pulses** to fill up the pipe and retrieve the first output from R5.

TABLE 9-1 Content of Registers in Pipeline Example

Clock Pulse Number	Segment 1		Segment 2		Segment 3
	R1	R2	R3	R4	R5
1	A_1	B_1	—	—	—
2	A_2	B_2	$A_1 * B_1$	C_1	—
3	A_3	B_3	$A_2 * B_2$	C_2	$A_1 * B_1 + C_1$
4	A_4	B_4	$A_3 * B_3$	C_3	$A_2 * B_2 + C_2$
5	A_5	B_5	$A_4 * B_4$	C_4	$A_3 * B_3 + C_3$
6	A_6	B_6	$A_5 * B_5$	C_5	$A_4 * B_4 + C_4$
7	A_7	B_7	$A_6 * B_6$	C_6	$A_5 * B_5 + C_5$
8	—	—	$A_7 * B_7$	C_7	$A_6 * B_6 + C_6$
9	—	—	—	—	$A_7 * B_7 + C_7$

Four-segment pipeline



- ♦ Any operation that can be decomposed into a sequence of sub operations
- ♦ The technique is efficient for those applications that need to repeat the same task many times with different sets of data.
- ♦ The operands pass through all four segments in a fixed sequence.
- ♦ Each segment consists of a combinational circuit S ,
- ♦ The segments are separated by registers R , that hold the intermediate results between the stages.

Segments in pipeline

S1								
S2								
S3								
S4								
Clock pulse	1	2	3	4	5	6	7	8



**Space-
time
Diagram**

- ◆ Suppose Processes can be executed in 4 steps so we have 4 segments
- ◆ These four steps are performed by S1, S2, S3, S4 separate segments and in sequence

Space timing diagram for process in non-pipeline

S1	p1				p2			
S2		p1			p2			
S3			p1			p2		
S4				p1			p2	
Clock pulse	1	2	3	4	5	6	7	8

- ◆ When we don't have parallel processing or pipelining, A process comes through S1 and goes out of S4
- ◆ Each stage/segment is passed in one clock pulse
- ◆ By the time, the other segments are idle (free)

Space timing diagram for process in pipeline

S1	p1	p2	p3	p4	p5			
S2		p1	p2	p3	p4	p5		
S3			p1	p2	p3	p4	p5	
S4				p1	p2	p3	p4	p5
Clock pulse	1	2	3	4	5	6	7	8

- ◆ So other process is kind-of executing parallely, since p1 is not fully executed but other has already started executing

Speedup & Equation of speedup for k-segment pipeline processing

- ♦ **Speedup:** It is the ratio of pipeline processing over a equivalent non-pipeline processing.
- ♦ Speedup mathematically is defined as

$$S = \frac{k \times n}{k + (n - 1)}$$

Where, **k** is no. of stages
n is no. of processes

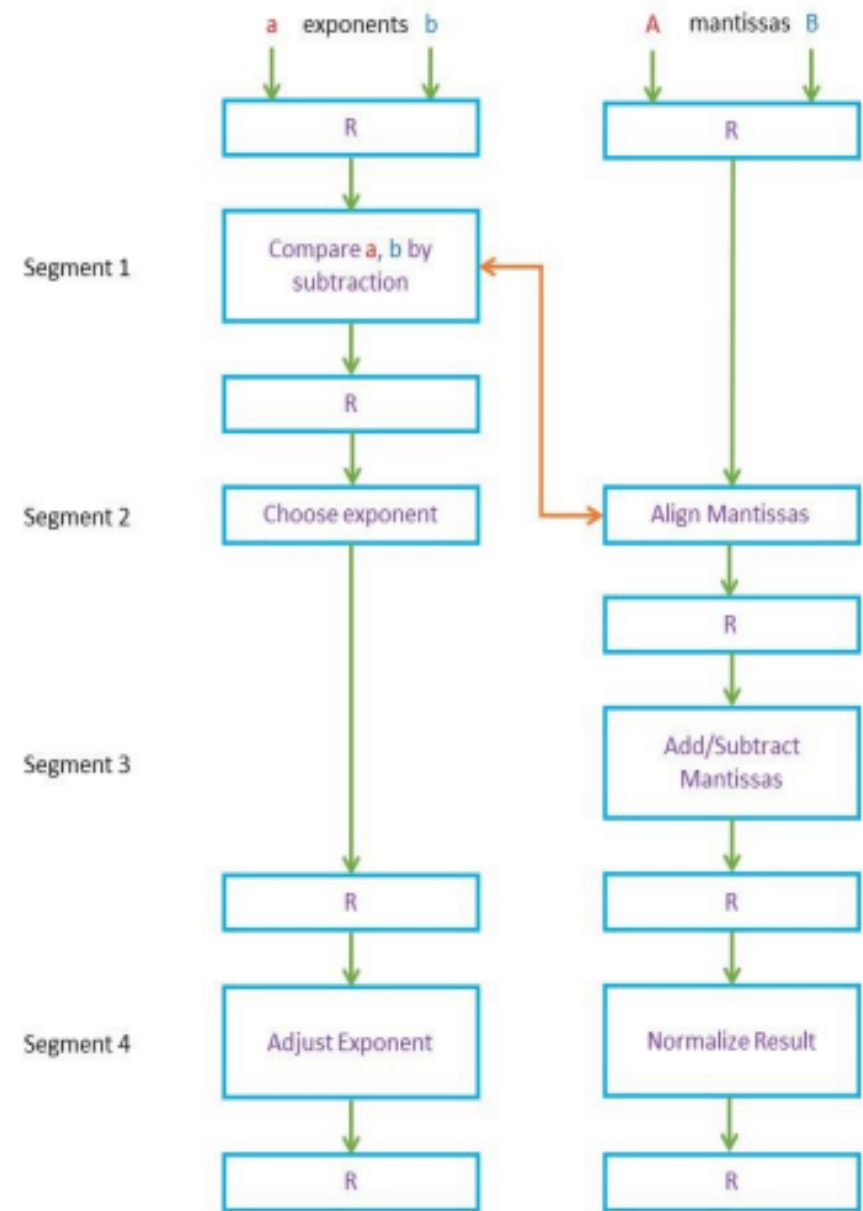
Non-pipeline takes $k \times n$ Clock cycles

Pipeline takes $k + (n - 1)$ Clock cycles

Theoretically, Maximum speedup is k

Arithmetic Pipeline

- ♦ The inputs to the floating-point adder pipeline are two normalized floating-point binary numbers.
 - ♦ $X = A \times 2^a$
 - ♦ $Y = B \times 2^b$
- ♦ A and B are two fractions that represent the mantissas and a & b are the exponents.
- ♦ The sub operations that are performed in the four segments are:
 - ♦ 1. **Compare the exponents.**
 - ♦ 2. **Align the mantissas.**
 - ♦ 3. **Add or subtract the mantissas.**
 - ♦ 4. **Normalize the result.**
- ♦ The exponents are compared by subtracting them to determine their difference.
- ♦ The larger exponent is chosen as the exponent of the result.



- ◆ This produces an alignment of the two mantissas.

- ◆ The two mantissas are added or subtracted in segment 3. The result is normalized in segment 4.

- ◆ Consider the two normalized floating-point numbers:

$$X = 0.9504 \times 10^3$$

$$Y = 0.8200 \times 10^2$$

- ◆ The two exponents are subtracted in the first segment to obtain $3 - 2 = 1$.

- ◆ The larger exponent 3 is chosen as the exponent of the result. The next segment shifts the mantissa of Y to the right to obtain

$$X = 0.9504 \times 10^3$$

$$Y = 0.0820 \times 10^3$$

- ◆ This aligns the two mantissas under the same exponent. The addition of the two mantissas in segment 3 produces the sum

$$Z = 1.0324 \times 10^3$$

- ◆ The sum is adjusted by normalizing the result so that it has a fraction with a nonzero first digit.

- ◆ This is done by shifting the mantissa once to the right and incrementing the exponent by one to obtain the normalized sum.

$$Z = 0.10324 \times 10^4$$

Thank You